Pitch — AI-Assisted Requirements Traceability and Software Specification Analysis

A guide for presenting to judges and teachers. Plain English. ~8 minute talk.

---

1. The hook (say this first)

> A developer opens a small pull request. It "cleans up" the refund API — removes one field, adds a nicer one. Every test passes. The code is merged. Six months later an auditor finds that refunds can now be charged twice, and the company is fined.
>
> Nothing was broken in the code. What broke was the **meaning**. The API stopped doing what the business rule asked for — and no tool on Earth today checks for that.
>
> We built the tool that does.

---

2. The problem

Software is written twice, in two different languages:

- **Business people** write requirements in plain English: "*Every refund must carry a unique transaction id so payments can be audited.*"
- **Engineers** write API specifications in a technical format (OpenAPI / Swagger JSON).

Over months and many versions, the API slowly drifts away from the requirement. We call this **specification drift**. It is dangerous because:

- It is **silent** — no test fails, no error appears.
- It **adds up** — many tiny changes become one big compliance failure.
- The two sides use **different words** — "transaction id" vs "idempotency key" vs "audit reference" — so simple keyword search cannot connect them.
- In banking, healthcare, and e-commerce, undetected drift means **audit failures and fines**.

Existing tools do not help:

Tool	What it checks	What it misses
Jira / DOORS traceability	Manual links	Whether the code actually does what the link claims
API linters (Spectral)	Is the file well-formed	Whether it matches the business rule
Keyword / TF-IDF search	Matching words	Same meaning, different words
Manual review	Everything, slowly	Does not scale to 100+ endpoints

---

3. The solution

We teach a computer to **compare meaning, not words**.

1. Read the requirements document → split into individual requirements.
2. Read the API specification → split into individual endpoints (with all their fields and descriptions).
3. Turn every requirement and every endpoint into a list of 768 numbers — a "meaning fingerprint" — using a transformer AI model (`all-mpnet-base-v2`).
4. Measure how close each requirement's fingerprint is to each endpoint's fingerprint (a score from 0 to 1). This builds a **traceability matrix**: requirements down the side, endpoints across the top, a score in every cell.
5. **Green** cell = the requirement is implemented. **Red** = it is not.

On top of that matrix we add four things:

- **Drift detection** — compare API v1 and v2 against the same requirements. If a score drops, we flag it and rate how serious it is (Minor / Major / Critical).
- **AI explanations** — for every problem, an LLM (GPT-4o-mini) writes three plain sentences: what changed, why it breaks the rule, and how to fix it. The LLM never decides the score — it only explains the number. So every result can be checked.
- **Pull request review bot** — simulates a GitHub bot that reads a code change, re-checks compliance, and posts "PASS" or "BLOCKED — here is the reason" before the code is merged.
- **Accuracy report** — we measure the system with standard information-retrieval metrics (Precision, Recall, F1, MAP) and compare it against a keyword baseline.

---

4. What we actually built (implementation)

This is not a slide-deck idea. It is a working system, delivered two ways.

A. Research pipeline + Streamlit dashboard

Seven Python modules (parser, embedder, similarity, drift detector, explainer, PR reviewer, evaluator) and a 5-tab dashboard: Traceability Heatmap, Drift Analysis, PR Reviewer, Evaluation Metrics, and a Vector Database Inspector.

B. Full production web app

- **Frontend:** React 18 with a custom dark/neon design, a 3D animated background, and 6 screens (landing, dashboard, traceability, drift, PR bot, benchmarks).
- **Backend:** FastAPI with **Google login**, secure JWT tokens, projects, file upload, and a **Qdrant vector database** that keeps every user's and project's data separate.
- If the vector database is switched off, the system automatically falls back to in-memory computation — it never crashes.

Numbers that prove it works

Check	Result
Automated tests	**59 / 59 passing**
Compliance detection on the demo	100% of 10 requirements traced to 20 endpoints

Check	Result
Our method vs keyword baseline (F1 score)	**0.71 vs 0.34** — roughly double
Ranking quality (MAP)	**1.00** — every requirement's correct endpoint ranked first
Drift detection	Caught **both** injected breaking changes (refund idempotency, 2FA removal)
PR bot	Correctly PASSED the safe change and BLOCKED both unsafe changes

We also proved it on a **second, unrelated domain** (a digital banking API) — the system correctly found that removing "step-up authentication" left a security requirement unimplemented, dropping compliance from 100% to 87.5%.

---

5. Use cases with real examples

Example 1 — The refund bug (our main story)

- **Requirement REQ-04:** refunds must carry a client-provided `transaction_id` for audit and to prevent double refunds.
- **The change:** a developer removes `transaction_id`, replaces it with a server-generated reference.
- **Our system:** the PR bot returns **BLOCKED**. The AI explanation says: "*The client `transaction_id` field was removed. The requirement needs a client-provided id for idempotent processing; a server id cannot prevent a retried request from creating a second refund. Fix: restore `transaction_id` as a required field.*"
- **Without our system:** merged silently, discovered in an audit months later.

Example 2 — The missing 2FA

- **Requirement REQ-09:** high-value orders (over $500) must pass two-factor authentication.
- **The change:** API v2 quietly drops the 2FA step from checkout.
- **Our system:** Drift page flags REQ-09 as drifted; compliance score falls; AI explains that the `two_fa_token` field and the enforcement text were removed.

Example 3 — The safe change (no false alarm)

- **The change:** add an *optional* `discount_code` field to the cart endpoint.
- **Our system:** PR bot returns **PASS** — no requirement lost alignment. This shows the tool does not cry wolf.

Example 4 — Different industry, same tool

- We ran it on a **banking API** it had never seen. It found that version 2 removed the step-up authentication endpoints, leaving "high-value transfers must use step-up auth" with no implementation. Compliance: 100% → 87.5%.

Who uses it

- **QA leads** — check every requirement is built before a release.
- **Architects** — audit an API redesign for hidden breakage.
- **Developers** — get compliance feedback on a pull request, before merge.
- **Auditors** — get an explainable, reproducible compliance record.

---

6. Why our approach is the right one

- **Meaning, not keywords** — a transformer model understands that "idempotency key" and "unique transaction id" mean the same thing.
- **Explainable, not a black box** — the score is always a reproducible math result; the AI only writes the explanation. An auditor can verify every number.
- **Built on API specifications** — the actual contract engineers ship, which most research ignores in favour of source code.
- **Shift-left** — it catches drift in the pull request, not in a post-release audit.
- **Extensible** — we designed a clean "Alignment Engine" so the same core can later compare *any* intent to *any* implementation (for example, a GitHub issue to the code that claims to implement it).

---

7. What is next

The next build is a **real GitHub App** (fully designed, in `docs/architecture/pr-review-bot-roadmap.md`):

- Runs automatically on every pull request.
- Reads the linked GitHub issue as the "intent" and the real code diff as the "implementation".
- Adds a code-aware AI model for reading source code.
- Posts a live review comment and a pass/fail check, and can block a merge.
- Proven with a labelled benchmark before release.

---

8. Research outcomes

- Addresses five documented gaps in the requirements-engineering literature (no end-to-end semantic verification, weak semantic drift detection, poor CI/CD integration, API specs ignored, no explainability).
- Targets a provisional patent and a Scopus-indexed conference/journal paper.
- All results reproducible from the synthetic dataset shipped with the code.

---

9. One-line summary

> **We built an AI system that reads business requirements and API specifications, measures whether the software actually means what the business asked for, explains every gap in plain language, and blocks non-compliant changes before they are merged.**

---

10. Demo script (2 minutes, live)

1. Open the dashboard → create a project → upload the SRS + API v1 + API v2.
2. Click **Run Analysis** → show the traceability matrix: 100% compliance, all green.
3. Open **Drift** → show REQ-09 flagged → click it → read the AI explanation aloud.

4. Open **PR Bot** → run "Remove refund idempotency" → show the **BLOCKED** result and the fix instruction.
5. Open **Benchmarks** → show F1 0.71 vs 0.34 → "every number here is reproducible".

Backup if offline: everything above also runs from `pytest -q` (59 passing tests) and the Streamlit app with the built-in dataset — no internet or API key required.